Generators and Decorators in Python

Both build on first-class functions: generators produce values lazily, decorators wrap behaviour.

Generator toolkit

Tool	Purpose
`yield x`	pause and produce a value
`next(g)` / `for x in g`	resume the generator
`(x for x in it)`	generator expression
`g.send(v)`	push a value into a `yield` expression
`g.throw(E)` / `g.close()`	inject an error / stop it
`yield from sub`	delegate to a sub-iterator
`return v`	end; value in `StopIteration.value`
`itertools`	`islice` `count` `cycle` `chain` `takewhile`

Decorator template

```
from functools import wraps

def decorator(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        # ...before...
        result = fn(*args, **kwargs)
        # ...after...
        return result
    return wrapper
```

Together 1 – decorating a generator function

```
def counted(gen_fn):
    @wraps(gen_fn)
    def wrapper(*a, **kw):
        n = 0
        for item in gen_fn(*a, **kw):
            n += 1
            yield item            # stay lazy
        print("yielded", n, "items")
    return wrapper

@counted
def evens(limit):
    for i in range(0, limit, 2):
        yield i

print(list(evens(7)))
```

OUTPUT

```
yielded 4 items
[0, 2, 4, 6]
```

Together 2 – generator as context manager

```
from contextlib import contextmanager

@contextmanager
def tag(name):
    print(f"<{name}>")
    try:
        # `with` body runs here
        yield
    finally:
        print(f"</{name}>")

with tag("p"):
    print("hello")
```

OUTPUT

```
<p>
hello
</p>
```

Together 3 – a pipeline of generators

```
def read(lines):
    for ln in lines:
        yield ln.strip()

def only_errors(lines):
    return (s for s in lines if s.startswith("E"))

def upper(lines):
    return (s.upper() for s in lines)

raw = ["E disk", "ok", "E net ", "ok"]
print(list(upper(only_errors(read(raw)))))
```

OUTPUT

```
['E DISK', 'E NET']
```

Gotcha – caching a generator

```
from functools import cache

@cache
def nums():
    # cached object is a generator
    return (n for n in range(3))

# 2nd call: exhausted!
print(list(nums()), list(nums()))
```

OUTPUT

```
[0, 1, 2] []
```

Problem	Fix
generator reused → empty	call the function again, or keep a list
decorator hides name / docstring	`@wraps(fn)`
decorated function returns `None`	`return fn(*args, **kwargs)`
`@cache` on a generator function	cache `tuple(...)`, not the generator
`send()` on an unstarted generator	`next(g)` first, or auto-prime